Prompt：我是 PHM 专家，有一与 ai 协作完成以下任务，任务详情如下，请为我出具六小时计划书。（任务详情）

AI:

一、任务与成功标准（前置于计划）

论文：*A Large-Scale Annotated Multivariate Time Series Aviation Maintenance Dataset from the NGAFID*（arXiv 可查，例如 ADS/arXiv 条目 arXiv:2210.07317 指向的预印本）。

数据：论文作者通常提供官方下载方式或配套代码（复现时以论文与项目页为准）；计划内预留"获取与校验数据"时间。

二分类定义（须在日志里写清你自己的界定）：

- 正类/负类分别对应"维护前航班"与"维护后航班"（或相反，但要固定并写入 README）。
- 明确是否排除、或单独处理"维护当日航班""同架机多次维护"等边界情况——这是评委常会问的"专家判断"点。

必须交付自检：

项	检查点
代码仓库	5 折交叉验证；每折准确率 + 均值±标准差
一键运行	`pip install -r requirements.txt` + 一条主入口命令
Git	≥5 次提交，能看出迭代（bugfix / 特征 / 模型 / 评估）
AI 协作日志	关键 Prompt、你的修正与主导、认知迭代证据

二、六小时总览（建议按块执行）

时段	主题	产出
第 1 小时	论文精读 + 数据落盘 + 问题形式化	标签规则文档草稿、数据目录结构
第 2 小时	数据接口与基线特征	可跑通的读取→标签→简单分类器
第 3 小时	五折 CV 与评估框架	每折指标 + 汇总统计
第 4 小时	复现思路对齐 + 一处"改进"	对比表（基线 vs 改进）
第 5 小时	可复现工程化	`requirements.txt`、配置、主脚本、README
第 6 小时	Git 历史与 AI 协作日志定稿	≥5 commits、日志成稿

Prompt：我已经把论文 pdf 格式放在了这个文件夹里面，现在请为我提炼论文的重点内容以及我实现这个项目所需要使用的信息，论文里面实现这个二分类检测所使用的模型

AI：

1. 论文核心要点（与你的课题直接相关）

数据与来源

- 来自 **NGAFID** 飞行记录 + **MaintNet** 维护记录；真实运行数据，非仿真。
- 规模（全文数据集）：约 **31,177 飞行小时 / 28,935 个航班**，相对 **2,111 次非计划维护**，维护问题聚类为 **36 类**。
- **每条航班**：23 个传感器，**每秒 1 条**多维时间序列；公开数据约 **4.3 GB、超 1 亿行**；机型均为 **Cessna 172**。
- 公开获取（文中写明）：Zenodo doi:10.5281/zenodo.6624956、Kaggle、以及 **GitHub 辅助下载与基准**：`https://github.com/hyang0129/NGAFIDDATASET`。

任务设定（论文表述）

- 论文将问题框成 **二元 RUL 相关**：关心 **P(RUL > x)**，基准子集中取 **x = 2 天**。
- **子集构造（基准用）**：只保留「维护前后样本量都 ≥ 50 航班」的类别 → **19 类**；子集中约 **维护后 5844 / 维护前 5602** 个航班。
- **标签与时间窗（3.2 节，子集）**：取 **维护期前后两天内**的航班，**排除维护进行中的航班**；
  - 原文写法：维护 **2 天内、维护前**的航班标为 **P(RUL>x)=0**；维护 **2 天后**的航班标为 **P(RUL>x)=1**（新件、期望短期内不坏）。

重要：第 **4.1 节「维护问题检测」里又用另一套符号**说明同一类二分类：**维护后 = 负类，维护前 = 正类**。
你实现时只要**自洽**（README 写清 0/1 或 pos/neg 对应哪一侧），并与论文表格含义对照即可，避免混用两套叙述。

困难点（写进你的 AI 日志/报告很有价值）

- 方差很大来自 **飞行员操作、航路、天气**等，不全是部件状态 → 与「纯异常检测」假设不同。
- 全数据 **36 类极度不平衡**；基准子集刻意削弱这一点。
- 约 **1% NaN**；全文数据**不替你填 NaN**，基准实验里再处理。

2. 你实现「维护前 vs 维护后」二分类需要落地的信息

项目	论文中的可操作说明
样本粒度	**每个航班 = 一条样本**（23 通道 × 变长时间序列）。
基准子集规则	≥50 航班/类；±2 天窗口；去掉维护中航班；19 类二元任务。
长度处理	**截取每个航班的最后 4096 个时间步**，短序列 padding 到相同长度。
归一化（基准）	**MinMax 到 [0,1]**；文中写 **最值用「全部数据」计算**——复现时若要更严谨，可在日志里讨论是否改为 **仅在训练折上拟合 scaler**（与原文严格一致则用全文最值）。
验证协议	**5 折交叉验证**（与 Table 2 一致）。
评估指标	论文报告 **Binary Accuracy**（及 train/val loss）；你的作业若要求每折准确率 + 均值±标准差，可直接对齐该设置。
代码起点	官方 **Colab / Jupyter** 与数据处理：`https://github.com/hyang0129/NGAFIDDATASET`。

3. 论文里「维护事件二分类（检测）」使用的模型

基准实验 Task 1：maintenance issue detection（维护前 vs 维护后）使用的模型为三类（HIVE-COTEv2 **因训练超 24h 未出结果**）：

模型	说明（据 4.2 节）	论文 Table 2 二元分类验证准确率（5 折平均）
ConvMHSA	1D 卷积把多变量序列压到更短序列，再堆 MHSA（**类 BERT encoder**），最后 GAP + 全连接分类；结构与 Yang et al. (2021) 一致。	76.0%
InceptionTime	采用 Fawaz et al. (2020) 的 Inception 结构；为省算力**未做 5 模型集成**，文中「InceptionTime」指 **单模型**。	75.5%
MiniRocket	Dempster et al. (2021)；实现为 **Oguiza (2022) 的 GPU 版**。	59.8%

论文中的训练配置摘要（与你复现对齐时很有用）

- **ConvMHSA / InceptionTime**：Google Colab **TPU v2-8**；200 epochs；200 steps/epoch；batch **128**；5-fold CV；Adam；学习率 **ConvMHSA 3e-5，InceptionTime 1e-4**。
- **MiniRocket**：**V100 GPU**；200 epochs；143 steps；batch **64**；Adam **2.5e-5**；同样截断/填充 **4096**。

Prompt:ConvMHSA Inception MiniRocket 为我介绍这三个模型的特点以及区别，我想复现维护事件二分类检测使用哪个模型你最推荐，你可以询问我目前软硬件的具体情况以及最后达到的准确率来综合判断

AI:

MiniRocket

- **思路**：用大量**固定、随机**的**一维卷积核**把时间序列扫一遍，得到很多特征（命中比例、位置等精简特征），最后在特征上做**线性分类器**（如 Ridge / Logistic）。
- **特点**：训练**极快**、占显存相对小、实现成熟（如 `sktime`、作者 GPU 实现）、**调参压力小**。
- **局限**：表达能力主要靠"随机卷积 + 线性头"，对**又长又多通道、且模式很细**的序列，可能**欠拟合**；你们论文里在该任务上 ~59.8%，明显低于两个深度模型。

InceptionTime（论文里用的是单路 Inception，未做 5 模型集成）

- **思路**：**多尺度 1D 卷积**（Inception 模块）+ **残差块**，叠到一定深度，再 **全局平均池化** + **全连接** 做分类。专为**时间序列分类**设计，在 UCR 等基准上很常见。
- **特点**：**端到端**学表示，对多尺度时间模式较友好；比 Transformer 类结构通常**更省算力、更稳定**一点。
- **局限**：序列很长时（你们论文里是截到 **4096 步 × 23 通道**）仍要跑不少卷积；要一定 GPU 更舒服；集成版更强但更贵。

ConvMHSA（卷积 + 多头自注意力）

- **思路**：先用 **1D CNN** 把序列**压缩**到较短长度（论文写的是 4096→512），再用多层 **Multi-Head Self-Attention** 建**长程依赖**，最后 **GAP + 全连接**。
- **特点**：CNN 管**局部/多通道融合**，Attention 管**全局时序关系**；在**复杂、长序列**任务上往往比纯 CNN 或纯 Rocket 更有上限。论文里在该任务上 ~**76.0%**，略优于单 Inception。
- **局限**：参数量与显存、训练时间通常 ≥ **Inception**；实现与调参（层数、头数、学习率）比 MiniRocket **重**。

2. 三者的核心区别（一把抓）

维度	MiniRocket	InceptionTime	ConvMHSA
范式	随机卷积特征 + 线性分类	纯卷积深度网络	CNN 下采样 + Transformer 式注意力
训练成本	低	中	中高
数据与任务难度	简单/中等序列往往够用	中长序列、多尺度模式	长序列、依赖关系复杂时更有优势
可解释/可维护	较好（特征固定、流程清晰）	中等	相对难
论文中该二分类准确率（参考）	~59.8%	~75.5%	~76.0%

本人决策：以「先跑通管线 + 时间紧 」为第一目标，而且设备没有显卡，用 MiniRocket 做快速 baseline

Prompt：使用论文 Section 3.2 中定义的基准子集（benchmark subset），即 19 类维护事件（19 maintenance-event subsets），帮我判断文件夹中的 2days 是我所需要的基准子集吗

AI：是的

Prompt：在本文件夹中编写相关 python 代码，使用 MiniRocket 模型跑五折交叉验证（5-Fold CV），报告每折准确率及均值±标准差，代码需一键运行（pip install -r requirements.txt）

AI：

新增文件

文件	作用
`requirements.txt`	`numpy`, `pandas`, `scipy`, `scikit-learn`, `numba`, `sktime`
`dataset.py`	与论文一致：每条航班 `(T,23)` → NaN 中位数填充 → **取最后 4096 步、不足则前部零填充** → `X` 为 `(23,4096)`；**按训练折**做通道 MinMax 到 [0,1]
`run_minirocket_cv.py`	读 `flight_header.csv` + `flight_data.pkl` → `MiniRocketMultivariate` + `RidgeClassifierCV` → 每折准确率 + mean +/- std
`run_experiment.bat`	`pip install -r requirements.txt` 后 `python -u run_minirocket_cv.py`

概念理解

什么是PHM

PHM 是 Prognostics and Health Management 的中文常称 **“预测与健康管理”**（也译“预后与健康管理”）。

它面向设备/系统在全寿命周期内的**健康状态监测、诊断、预测与维护决策**，典型目标包括：

- **诊断（Diagnostics）**：现在是否正常、哪里出了问题。
- **预测（Prognostics）**：劣化或故障**会怎样发展**，例如 RUL（**剩余使用寿命**）、失效概率、还有多久需要维护。
- **健康管理（Health Management）**：基于健康信息做 **运维策略**（何时检、何时换件、如何降风险/降成本）。

你做的“维护前 vs 维护后航班”二分类，就是 PHM 里与**真实运维记录对齐**的一类数据驱动任务，用来支撑**维护需求/状态相关**的建模与评估。

RUL（剩余使用寿命）是什么

RUL（Remaining Useful Life，剩余使用寿命） 在 PHM 里一般指：在**当前观测到的状态**和**给定的使用/任务条件**下，系统或部件**还能继续安全、合格地服役多长时间**（或多少次循环、多少里程等），直到达到**失效判据**或**维护/更换门槛**为止。

需要把握几点：

1. **“寿命”指的不是日历寿命的唯一概念**
   常用的是 **到失效/到不可用** 还剩下的 **时间、循环次数、运行小时** 等单位，取决于设备和行业约定。
2. **必须绑定失效或维修定义**
   RUL 不是绝对常数，它依赖于 **什么叫坏**（性能下降到某阈值、裂纹到某尺寸等）以及 **是否在维护后重置**（换件、大修后 RUL 重新计算）。
3. **预测的是“剩余量”，不是“已用量”**
   和“已运行多久”相对，RUL 回答的是 **还能用多久**（或概率意义下的剩余时间分布）。
4. **和你这篇 NGAFID 论文的关系**
   论文里把问题框成与 **P(RUL > 某阈值)** 相关的**二元表述**（例如与“维护前后 2 天窗口”结合），本质是在用维护记录构造**与部件健康/临近维护相关的标签**，用飞行传感器序列去估计**这种 RUL 相关信息**；并不是每个航班都带有精确的连续 RUL 数值标签，而是**用维护事件在时间上对齐**的一种工程化定义。

一句话：**RUL 回答“还能安全用多久”，定义离不开失效/维护标准和数据如何标注。**

所以说RUL 主要发挥“问题定义与解释桥梁”的作用，而不是在训练里逐条回归一个真实 RUL 数值。定义任务、解释标签含义、对齐 PHM 文献，而不是一个被直接监督预测的目标变量

可以这么理解，你的表述是准确可用的。

再补一句**边界**，避免答辩时被抠字眼：

- **在你们这条基准二分类主线里**：监督目标是 `before_after`（**维护前/后**）这类**离散标签**；**RUL 是论文用语境下的“语义解释/问题动机”**，用来说明你分的两类在 PHM 里对应什么样的健康状态差异。
- **并不是**：数据里一定逐航班给了“真实 RUL=xx 小时”的连续目标，然后你们用 MSE 去回归它（除非你自己额外定义/构造这种标签并改任务）。

如果你把这段话写进报告，我建议最后加半句更严谨：

> *“本工作的监督信号来自维护时间窗对齐规则；RUL 表述用于对齐 PHM 文献与解释类别语义，而非本基准中默认的连续回归目标。”*

问题以及解决方法：

为什么你的训练结果准确率达到0.76，论文只有0.59，出现了什么问题

使用的是minirocket

在你**确认用的是** MiniRocket 的前提下，若验证准确率仍在 **~0.76** 而论文 MiniRocket 约 **0.60**，按优先级可以这样理解：**更大概率是"评估设置/对比方式"和论文不一致**，而不是 MiniRocket 突然显著变强。

1. 先排除"看错模型 / 看错输出"

请自检：

- 特征提取是不是 `MiniRocketMultivariate`，分类头是不是 `RidgeClassifierCV`（**或别的线性分类器**），**没有**再叠一层深度网络。
- 打印的数是 **验证折（validation / fold-held-out）** 的 accuracy，**不是**训练集 accuracy。

2. 最常见原因：划分方式不是作者那一套

论文配套数据在 `flight_header.csv` 里有 `fold` **列**。若你用过 `StratifiedKFold(shuffle=True)` 这种随机分层划分，**很容易出现比官方 `fold` 划分高一截的结果**（同源维护/相近航班信息更容易"泄漏"到验证表现里）。

自检命令/设置：应使用 `--cv-mode fixed`（按 `fold` 列：train= `fold!=k`，val= `fold==k`），再报告 5 折均值。

3. 第二常见：用单折高分对比论文 5 折均值

论文 Table 2 是 **5 折平均**。你可能某一折碰巧到 0.76。
请贴：**5 折每一折** + mean ± std；若均值回到 ~0.55–0.65 一带，就和论文更像。

定位到主要原因：flight_header.csv 含官方 5 折的 fold 列，而当前脚本使用 StratifiedKFold(shuffle=True)，与作者仓库不一致，易造成与维护事件/机型相关的“软泄漏”，使验证准确率虚高。正在把默认交叉验证改为使用官方的 fold 列，并添加与官方一致的填充方式（短航班在序列末尾补零）及可选的 stats.csv 全局 MinMax。　最后得到合理的结果

加入了 Balanced accuracy（平衡准确率），与 F1 以及 ROC-AUC 三个指标

2. Accuracy / Balanced accuracy / F1 / ROC-AUC（每折返回值）

```
probs, targs = learner.get_preds(ds_idx=1)
y_true = targs.cpu().numpy().astype(np.int64).ravel()
p = probs.cpu().numpy()
y_pred = p.argmax(axis=1).astype(np.int64)
pos_score = p[:, 1] if p.ndim == 2 and p.shape[1] > 1 else p.ravel()
fold_accuracy = float(accuracy_score(y_true, y_pred))
balanced_acc = float(balanced_accuracy_score(y_true, y_pred))
f1 = float(f1_score(y_true, y_pred, average="binary", zero_division=0))
try:
    roc_auc = float(roc_auc_score(y_true, pos_score))
except ValueError:
    roc_auc = float("nan")
...
return {
    "accuracy": fold_accuracy,
    "balanced_accuracy": balanced_acc,
    "f1": f1,
    "roc_auc": roc_auc,
}
```